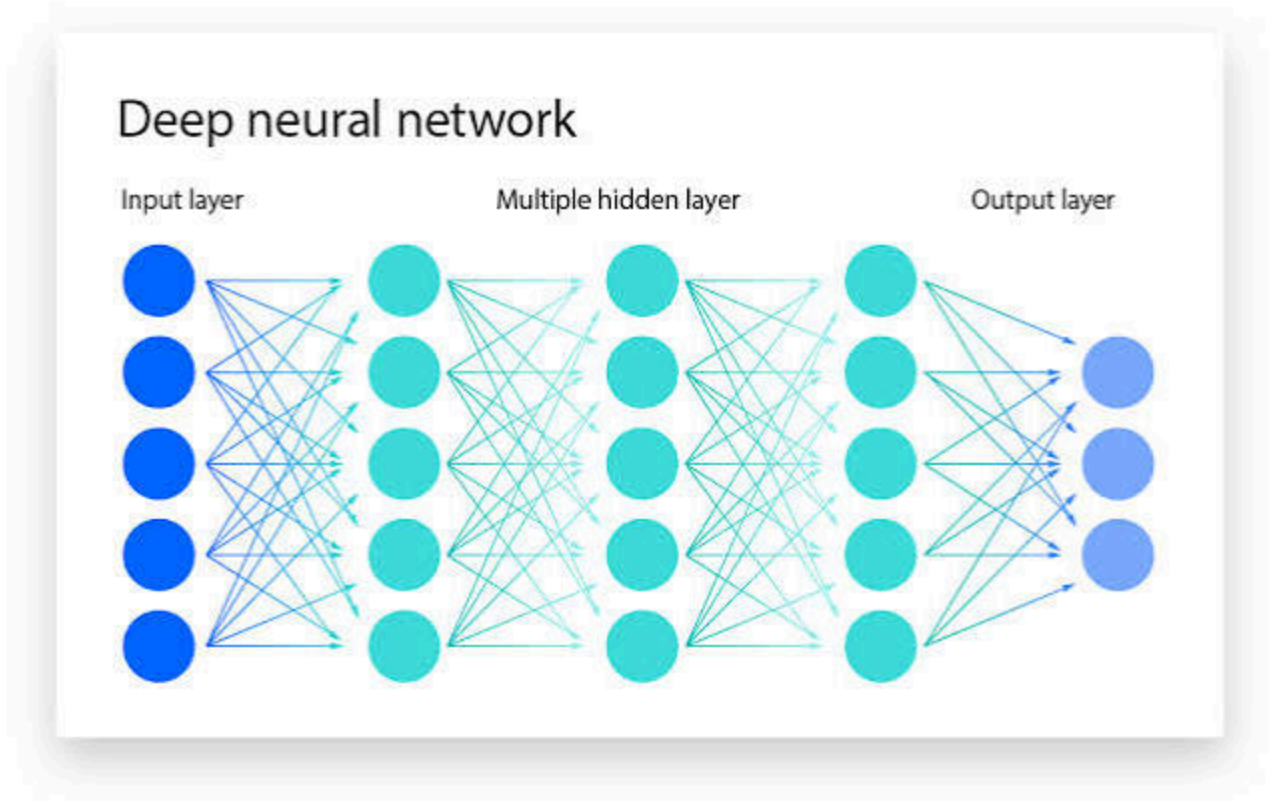


유연화 - PSnn

Person Sports Neural Network



7월 13일

데이터 수집

- 샘플 데이터



데이터셋 이름 : Pedestrians Videos

링크 : <https://www.kaggle.com/datasets/dsptlp/pedestrians-videos>

```
1  import kagglehub
2
3  # Download latest version
4  path = kagglehub.dataset_download("dsptlp/pedestrians-videos")
5
6  print("Path to dataset files:", path)
```

계획

1. 체육관 영상에서 구역을 나눠서 구역에 들어온 사람수로 판별
2. 전체 인원 - 농구인원(농구공 주변 인원이라 정확함) = 배드민턴 인원
3. 공 주변 사람 수를 가져오도록 구현
4. 그냥 옰로 모델 말고 옰로-포즈 모델로 학습 좀 더 시키기

샘플 데이터

폴더 구성 확인

```
1  from pathlib import Path
2
3  path = Path("폴더경로")
4
5  for p in sorted(path.iterdir()):
6      kind = "폴더" if p.is_dir() else "파일"
7      print(kind, "|", p.name)
```

출력 결과

파일		2954065-hd_1920_1080_30fps.mp4
파일		4791196-hd_1920_1080_30fps.mp4
파일		6896028-hd_1920_1080_30fps.mp4
파일		coco.names
파일		sample_image.jpg
파일		yolov3.cfg
파일		yolov3.weights

```
1  import os
2  import cv2
3  from huggingface_hub import hf_hub_download
4  from ultralytics import YOLO
5
6  # 설정
7  ## 입력 영상 경로
8  INPUT_VIDEO_PATH = "sampledata/2/2954065-hd_1920_1080_30fps.mp4"
9  ## 결과 저장 폴더 경로
10 OUTPUT_DIR = "sampleresult"
11
12 ## 결과 파일명
13 OUTPUT_FILENAME = "result.avi"
14
15 ### 처리 중 화면 미리보기 여부 (창 띄우기 싫으면 False)
16 SHOW_WINDOW = True
17 ### 확신도
18 CONF_THRESHOLD = 0.4
19
20 # 모델
21 REPO_ID = "Ultralytics/YOLO26"
22 FILENAME = "yolo26n.pt"
23 # COCO 기준 person 클래스 번호
24 PERSON_CLASS_ID = 0
25
26 def main():
27     # 모델 로드
28     weights_path = hf_hub_download(repo_id=REPO_ID, filename=FILENAME)
29     model = YOLO(weights_path)
30
31     # 입력 영상 열기
32     if not os.path.exists(INPUT_VIDEO_PATH):
33         raise FileNotFoundError(f"입력 영상을 찾을 수 없습니다: {INPUT_VIDEO_PATH}")
34
35     cap = cv2.VideoCapture(INPUT_VIDEO_PATH)
36     if not cap.isOpened():
37         raise RuntimeError(f"영상을 열 수 없습니다: {INPUT_VIDEO_PATH}")
38
39     fps = cap.get(cv2.CAP_PROP_FPS) or 30
40     frame_w = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
41     frame_h = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
42     total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
43
44     # 결과 저장 영상 준비
45     os.makedirs(OUTPUT_DIR, exist_ok=True)
46     output_path = os.path.join(OUTPUT_DIR, OUTPUT_FILENAME)
47
48     fourcc = cv2.VideoWriter_fourcc(*"mp4v")
49     writer = cv2.VideoWriter(output_path, fourcc, fps, (frame_w, frame_h))
50
51     print(f"입력 영상: {INPUT_VIDEO_PATH} ({frame_w}x{frame_h}, {fps:.1f}fps, 총 {total_frames}프레임)")
52     print(f"결과 저장 위치: {output_path}")
53     print("처리 중... 중간에 멈추려면 미리보기 창에서 'q'를 누르세요.")
54
55     frame_idx = 0
56     max_person_count = 0
57
58     while True:
59         ret, frame = cap.read()
60         if not ret:
61             break
62
63         frame_idx += 1
64
65         # person 클래스만 탐지
```



```

66     results = model.predict(frame, classes=[PERSON_CLASS_ID], conf=CONF_THRESHOLD, verbose=False)
67     boxes = results[0].boxes
68     person_count = len(boxes) if boxes is not None else 0
69     max_person_count = max(max_person_count, person_count)
70
71     if boxes is not None:
72         for box in boxes.xyxy.cpu().numpy():
73             x1, y1, x2, y2 = box.astype(int)
74             cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)
75
76     cv2.putText(
77         frame,
78         f"People: {person_count}",
79         (10, 40),
80         cv2.FONT_HERSHEY_SIMPLEX,
81         1.0,
82         (0, 255, 255),
83         2,
84     )
85
86     writer.write(frame)
87
88     if SHOW_WINDOW:
89         cv2.imshow("Person Count (processing)", frame)
90         if cv2.waitKey(1) & 0xFF == ord("q"):
91             print("사용자에 의해 중단되었습니다.")
92             break
93
94     if frame_idx % 30 == 0:
95         print(f" {frame_idx}/{total_frames}")
96
97     cap.release()
98     writer.release()
99     if SHOW_WINDOW:
100         cv2.destroyAllWindows()
101
102     print(f"완료. 최대 인원수: {max_person_count}명")
103     print(f"결과 영상 저장됨: {output_path}")
104
105
106 if __name__ == "__main__":
107     main()

```

- 1번 영상



2번 영상



3번 영상



잘되는 모습

저장

△ Audio extraction failed. Try opening in external player.

result.avi

People: 4

00:00 / 00:29

1x HD

문제점

멀리 있는 사람 인식이 안됨

- 평균 탐지 인원수

1번 영상

	n	s	m	l	x
640	6.18	7.69	8.72	8.73	7.87
960	10.00	12.31	16.59	14.27	12.93
1280	14.50	18.55	23.79	23.29	25.73

2번 영상

	n	s	m	l	x
640	6.24	8.11	9.16	9.21	9.61
960	8.15	9.38	9.96	10.13	10.31
1280	8.60	9.87	10.41	10.57	10.67

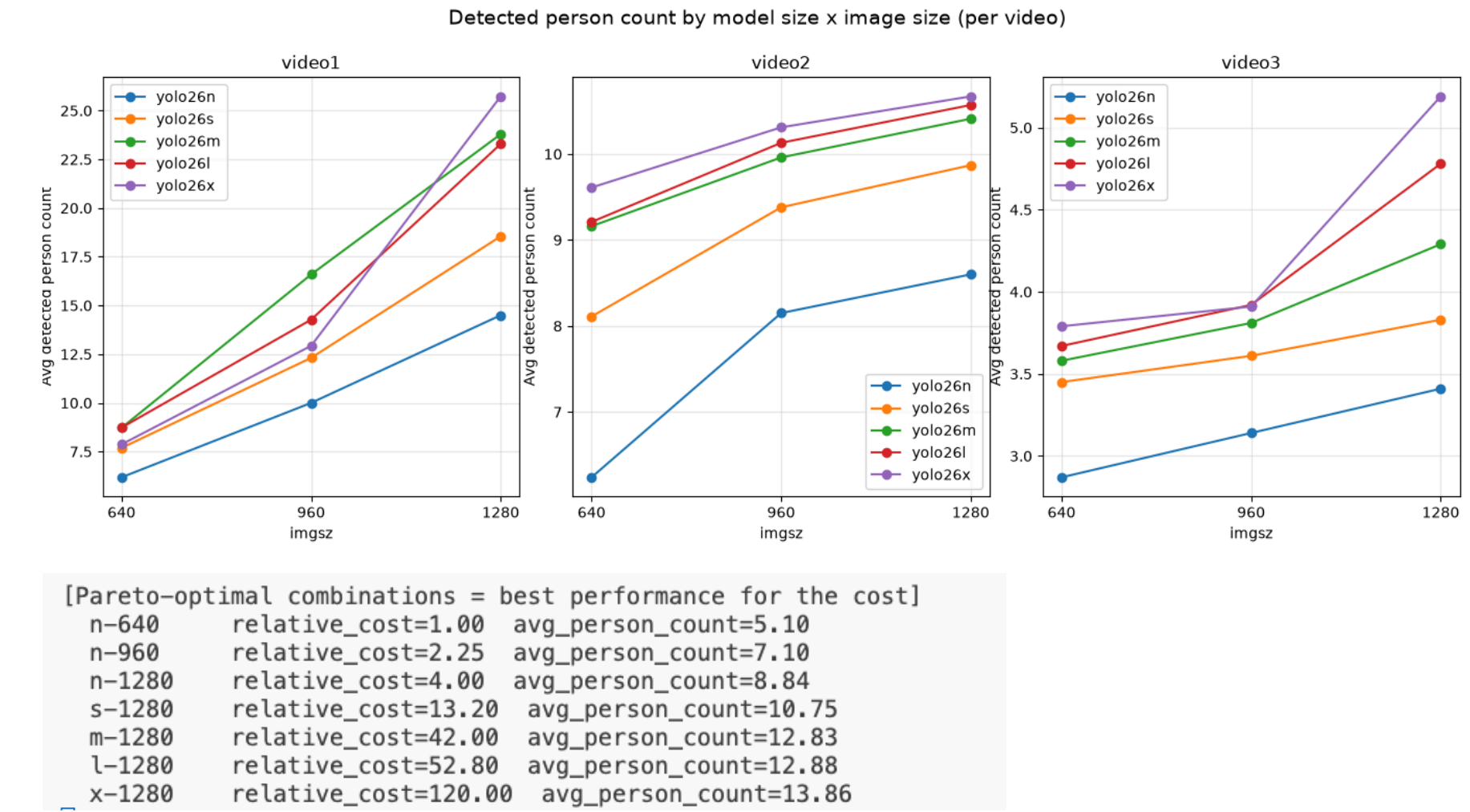
3번 영상

	n	s	m	l	x
640	2.87	3.45	3.58	3.67	3.79
960	3.14	3.61	3.81	3.92	3.91
1280	3.41	3.83	4.29	4.78	5.19

이 결과를 통해 최고 성능 모델 대비 성능이 떨어지는 것과 화질vs모델 성능을 비교하였다.
모델의 성능&화질이 좋아질 수록 멀리있는 사람들까지 인식 가능하다는 것을 알게되었으며

모델과 화질 중 무엇이 효율적이고 타당한 선택인지를 고르게 되었다.
좀 더 좋은 선택을 위해 데이터셋 내부의 3개의 영상 각각 벤치마크를 돌려보았다.

- 성능 변동 그래프

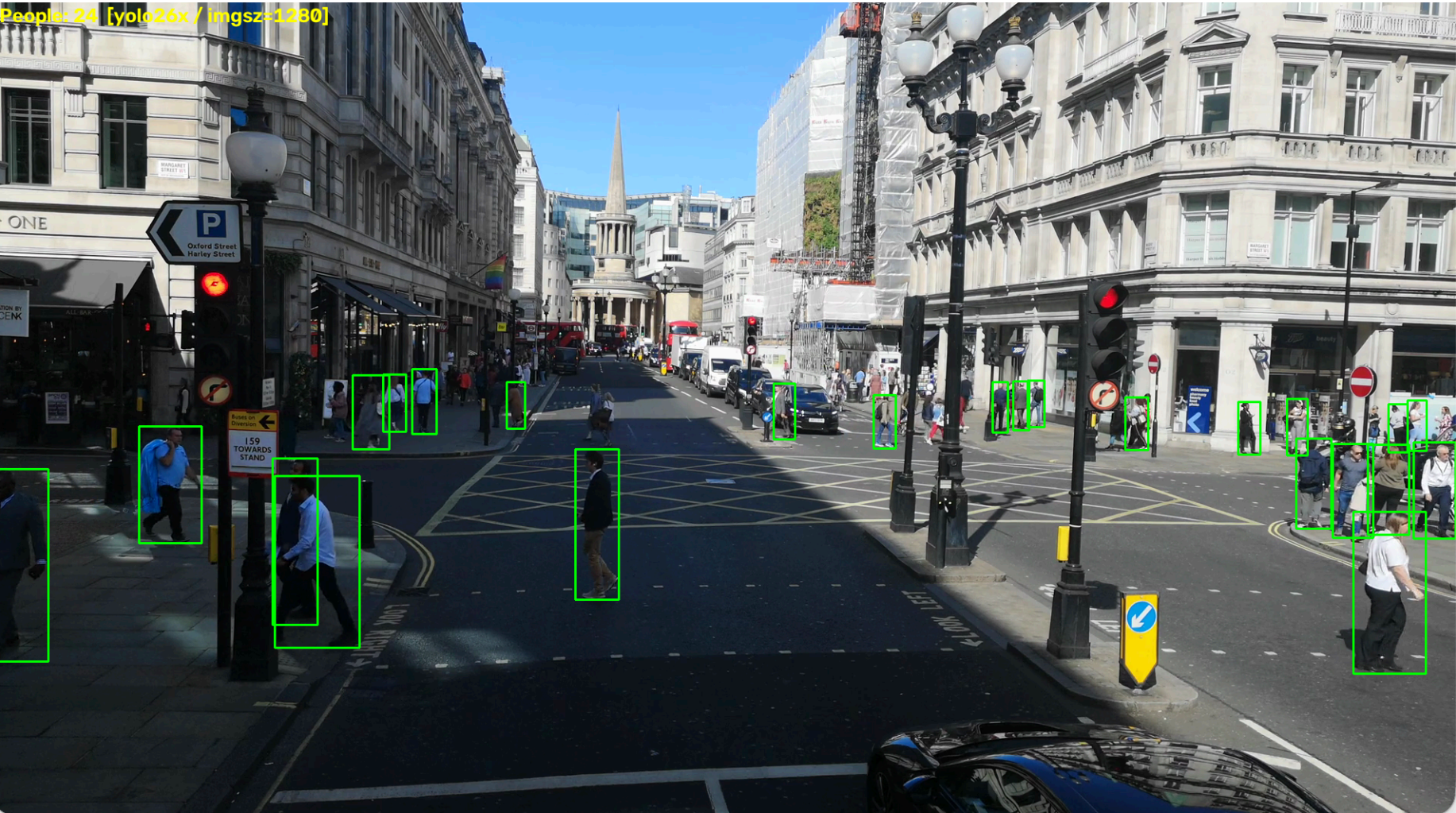


결론적으로 입력 이미지 사이즈 자체를 늘리는 것이 가장 효율적이며 이 후에 모델 성능을 결정하는 것이 타당하다
-> 같은 이미지 사이즈에서 모델 성능을 하나 올리는 거 대비 이미지 입력 사이즈를 한단계 올리는 것이 압도적으로 좋음

결론적으로 YOLO26에서는 이미지 입력 사이즈를 최대로 하고 모델 성능을 하드웨어 기기에 맞게 선택하도록함

최고 사양(x)&최고 화질(1280)

People: 24 [yolo26x / imgs<1280]



PSNN

> .venv

▼ sampledata / 2

2954065-hd_1920_1080_30fps.mp4

4791196-hd_1920_1080_30fps.mp4

6896028-hd_1920_1080_30fps.mp4

coco.names

sample_image.jpg

yolov3.cfg

yolov3.weights

> sampleresult

> sampleresult1

.python-version

benc

benchmark.py

graph.py

pyproject.toml

README.md

sampledata.py

sampledatas.py

samplemodel.py

samplemodelf.py

uv.lock

+

+

↺

📄

내일 할 꺼

우선 사람 샘플 데이터에 대한 결과를 바탕으로

웹사이트 api로 영상 보내서 처리하도록
오늘 찍은 운동 영상 데이터셋 활용 방안

각 방안에서의 타협점